



Taller Recuperativo

Objetivos

- Diseñar algoritmos para trabajar con Heaps
- Comprender en que casos es útil el uso de Heaps.

Introducción

En este taller recuperativo tu principal objetivo sera trabajar con Heaps e implementar algunas operaciones en heaps para luego resolver un problema teorico utilizandolas. Para esto, puedes considerar que ya cuentas con la implementacion basica de heaps, lo que incluye las funciones fundamentales tales como `insert`, `siftup`, `extractRoot`, etc

Parte a) Implementacion de Heap Sort

Para este ejercicio, trabajarás con la implementación clásica del algoritmo **Heap Sort**. Tu tarea será implementar el evento `HEAPSORT {N}`, donde `N` corresponde a la cantidad de elementos del arreglo que deberás ordenar.

Luego del evento, recibirás una línea con los `N` enteros del arreglo.

El algoritmo **Heap Sort** permite ordenar un arreglo utilizando la estructura de **Max-Heap**. Para esto, primero se transforma el arreglo en un heap válido y luego se extraen los elementos máximos uno a uno, dejándolos en su posición final.

Tu tarea será implementar en `C` las siguientes funciones:

```
void Siftdown(int arr[], int n, int i);  
void heapSort(int arr[], int n);
```

Donde:

- `arr` es el arreglo de enteros.
- `n` es el tamaño actual del heap.
- `i` es el índice del nodo desde donde se aplicará `Siftdown`.

El algoritmo debe seguir los siguientes pasos:

1. Leer los `N` elementos del arreglo.
2. Implementar `Siftdown`. Para esto:
 - (a) Considera el índice `i` como la raíz del subárbol actual.

- (b) Calcula el índice del hijo izquierdo como $2*i + 1$.
- (c) Calcula el índice del hijo derecho como $2*i + 2$.
- (d) Compara la raíz con sus hijos válidos y encuentra el mayor.
- (e) Si el mayor no es la raíz, intercambia ambos valores.
- (f) Luego, aplica nuevamente **Siftdown** sobre el índice donde quedó el valor intercambiado.

3. Construir el **Max-Heap**. Para esto:

- (a) Comienza desde el último nodo interno del arreglo.
- (b) Este índice corresponde a:

$$n/2 - 1$$

- (c) Desde ahí, aplica **Siftdown** hacia atrás hasta llegar a la raíz.

4. Ordenar el arreglo. Para esto:

- (a) Intercambia la raíz del heap con el último elemento del heap actual.
- (b) Reduce el tamaño del heap en una posición.
- (c) Aplica **Siftdown** desde la raíz para restaurar la propiedad de **Max-Heap**.
- (d) Repite este proceso hasta que el arreglo quede ordenado.

5. Imprimir el arreglo ordenado de menor a mayor.

El resultado esperado debe mostrar el siguiente output:

Por ejemplo, para el siguiente input:

input:	
1	HEAPSORT 6
2	9
3	4
4	1
5	7
6	3
7	2

se espera el siguiente output:

output:	
1	Arreglo ordenado via Heapsort:
2	1
3	2
4	3
5	4
6	7
7	9

Evaluación: La nota de la parte a) de tu taller es calculada a partir de testcases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto. Se entregan testcases públicos para que puedas probar tu implementación durante el taller.

Parte b) Ejercicio Teórico: Ordenamiento de un Arreglo k -Ordenado

Considere el siguiente concepto: un arreglo se dice **k -ordenado** si cada elemento está a lo más k posiciones de distancia de su posición final en el arreglo completamente ordenado. Por ejemplo, en un arreglo 2-ordenado, ningún elemento se encuentra a más de 2 posiciones de donde debería estar una vez que el arreglo está completamente ordenado. Se entrega un ejemplo de un arreglo 2-ordenado a continuación:

[2, 1, 4, 3, 6, 5, 8, 7]

Su objetivo es diseñar un algoritmo que, aprovechando la implementación de **Siftdown** desarrollada en la parte a), ordene un arreglo k -ordenado de n elementos en tiempo $\mathcal{O}(n \log k)$. **Hint:** Considere utilizar un heap auxiliar de tamaño acotado por $k + 1$.

En su respuesta debe detallar lo siguiente:

- **Diseño de la Estructura:** ¿Qué estructura de datos usarías para resolver este problema y por qué? Fundamente su respuesta.
- **Algoritmo Paso a Paso:** Describa brevemente cómo funciona el algoritmo: cómo empieza, cómo avanza, cómo termina y por qué cumple con lo pedido.
- **Análisis de Complejidad:** Justifique por qué el algoritmo garantiza una complejidad máxima de $\mathcal{O}(n \log k)$. Considere tanto el costo de cada operación de inserción y extracción sobre el heap auxiliar, como el número total de operaciones realizadas para los n elementos del arreglo.

La parte b) del taller se evaluará de manera manual. Por lo que se espera que el estudiante pueda explicar con sus propias palabras, ya sea en pseudocódigo o en un párrafo, cómo resolvería el problema.

Bonus: Implementar el evento K-SORT {N} {K}

Dada su solución de la parte b), impleméntela en C.

En este evento recibirás un arreglo k -ordenado de N elementos. Tu tarea será ordenarlo de menor a mayor utilizando un heap auxiliar, de modo que la solución tenga complejidad máxima $\mathcal{O}(N \log K)$.

Luego del evento, recibirás una línea con los N enteros del arreglo a ordenar.

El resultado esperado debe mostrar el siguiente output:

input:	
1	K-SORT 8 2
2	2
3	1
4	4
5	3
6	6
7	5
8	8
9	7

se espera el siguiente output:

output:

```
1 | Arreglo 2-ordenado ordenado:
2 | 1
3 | 2
4 | 3
5 | 4
6 | 5
7 | 6
8 | 7
9 | 8
```

Evaluación (Solo parte evaluada)

La nota de tu taller es calculada a partir de testcases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán testcase publicos para que puedas probar tu implementación durante el taller.

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 3 segundos** y utilizar menos de 1 GB de RAM¹, de lo contrario recibirás 0 puntos en ese test. Por último, está **estrictamente prohibido** el uso de la función `realloc`, cuyo uso implicará un descuento de 5 décimas.

La parte teórica del taller será revisada manualmente por el equipo docente. Se espera que el estudiante pueda justificar correctamente sus decisiones de diseño, explicar el funcionamiento de su algoritmo y fundamentar su análisis de complejidad utilizando sus propias palabras.

Integridad académica e IA

Cualquier acto deshonesto o fraude académico esta estrictamente prohibido. Se prohíbe el uso de herramientas de inteligencia artificial y el uso o manipulación de otros dispositivos electrónicos como laptops, tablet, teléfonos durante la evaluación.

Proceso de Entrega

Conexión a internet

Una vez iniciado sesión en el escritorio, seleccionar el icono de redes arriba a la derecha. "*availablenetwork*" → "*eduroam*". Para conectarse a eduroam, deben tener la siguiente configuración:

- Tener seleccionado la opción "No CA certificate is required".
- En la opción de "*Inner authentication*" seleccionar "*PAP*".
- Luego debe ingresar tus credenciales usuario uc. **IMPORTANTE:** en el usuario debes incluir el @, no solo el nombre.
- Ejemplos correctos: "*nombre@uc.cl*"; "*nombre@estudiante.uc.cl*", Incorrecto: "nombre" sin el @.

Descargar taller

Para descargar el código base que con el que trabajaran, deben abrir una terminal, estar conectados a internet y ejecutar `descargar-taller nombre_archivo.zip`

Donde el *nombre_archivo* debe ser reemplazado con el nombre que se avisara al inicio de cada taller.

¹Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`

Descomprimir archivo

Una vez descargado el archivo del taller deben descomprimirlo usando: `unzip nombre_archivo.zip`

Editor de código

El editor de código de la imagen se llama "*lite - xl*". Para abrir su trabajo tienen varias alternativas, las recomendables son:

- Abrir el editor de código, buscar la sección de "*file*" y seleccionar "*Open*" y buscar la carpeta donde quieres trabajar.
- Ir al explorador de archivos, buscar la carpeta donde quieres trabajar, hacer click derecho en la carpeta y seleccionar "*Openwith...*" y seleccionar el editor.
- **IMPORTANTE:** El editor no tiene autoguardado, por lo que cada vez que hagan cambios deben usar "*CTRL + S*" para guardar. Si ustedes no guardan sus archivos e intentan compilar, el proceso de compilado no guarda automáticamente, por lo que su código se compilara con la ultima versión que hayan guardado, **DEBEN GUARDAR ANTES DE COMPILAR.**

Espacio de trabajo

Al abrir el explorador de archivos o una terminal, verán una carpeta llamada código. Es **FUNDAMENTAL** que todo el trabajo que hagan lo tengan dentro de esa carpeta. Eso les asegura que en caso de que la imagen falle y tengan que reiniciar el computador, todo lo que está guardado en esa carpeta se mantendrá.

Uso del entorno de trabajo

En su carpeta de trabajo tendrán al menos los siguientes elementos:

- src: carpeta con su código
- makefile: archivo para compilar
- tests: carpetas de tests publicas
- run.sh: script para ejecutar todos los tests

Para compilar, en una terminal usen:

```
make
```

Para usar el script que ejecuta todos los tests, usan:

```
./run.sh
```

El run.sh genera una carpeta llamada outputs que guarda para cada test, el output de su programa. **Si desean ejecutar un único test** ocupan

```
./ejecutable ruta_al_input_del_test.txt archivo_output.txt
```

Donde ejecutable es el nombre del archivo que se genera al compilar, puedes revisarlo en las carpetas dentro de src, ejemplo de ejecutables: "pokemones", "magnemites". Ejemplo de ruta al archivo de input: tests/F1/pokemones/easy-0/input.txt y output.txt es un archivo txt que crean y es donde ira su salida.

En caso de querer debugear algún error de un test en especifico usar **valgrind**

```
valgrind ./ejecutable ruta_a_input.txt output.txt
```

Entrega Taller

Para entregar, también es importante que estén conectados a internet, entrar en una terminal, posicionarse en la ruta donde esta la carpeta de su entrega (no dentro de la carpeta misma). Finalmente, ejecutar

```
entregar-taller nombre_carpeta TX nAlumno1_nAlumno2_...
```

Donde: "*nombre_carpeta*" es el nombre de la carpeta que quieren entregar, por ejemplo: "*T2_evaluado_G2*". "*TX*" donde X corresponde al taller que se esta entregando: por ejemplo: "T2". Y *nAlumno1_nAlumno2_...* que son los números de alumnos de los integrantes del grupo que trabajaron separados por "_" (Si están realizando este taller de manera solitaria, solamente escriben su numero de alumno).
Ejemplo Correcto:

```
12345678_23456789 20387645 20387645_12345678_23456789
```

Ejemplo Incorrecto:

```
1234567823456789 Nombre1_12345678 Nombre1_Nombre2_Nombre3
```